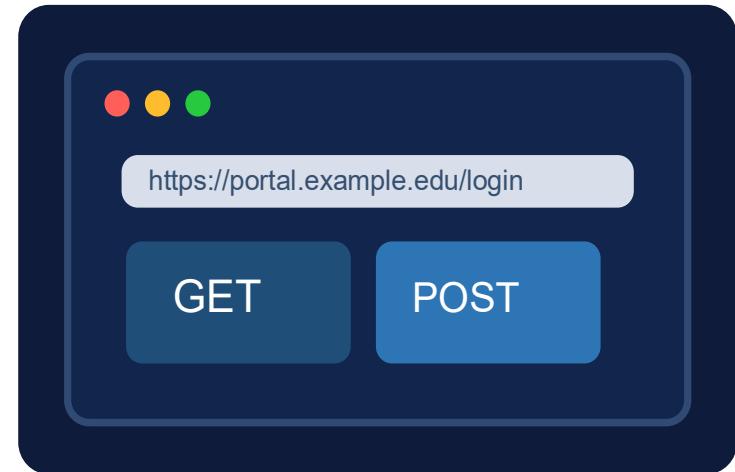


Lecture 4

Web Basics, HTTP, URLs, and Status Codes

Backend Python + Django Course
Second Year - Faculty of Computers & AI

Dr. Mohamed Nour Eldien



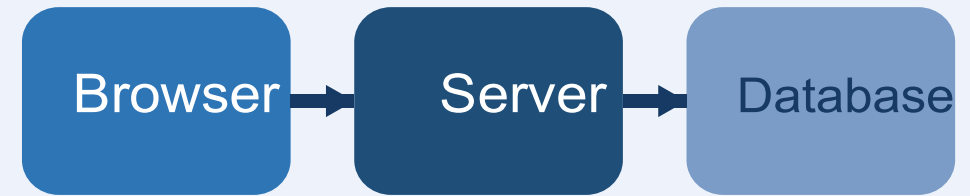
Lecture Focus

How a browser talks to a server, how URLs are built, and how HTTP responses are interpreted.

Learning Outcomes

By the end of this lecture, students should be able to explain the full basic web flow

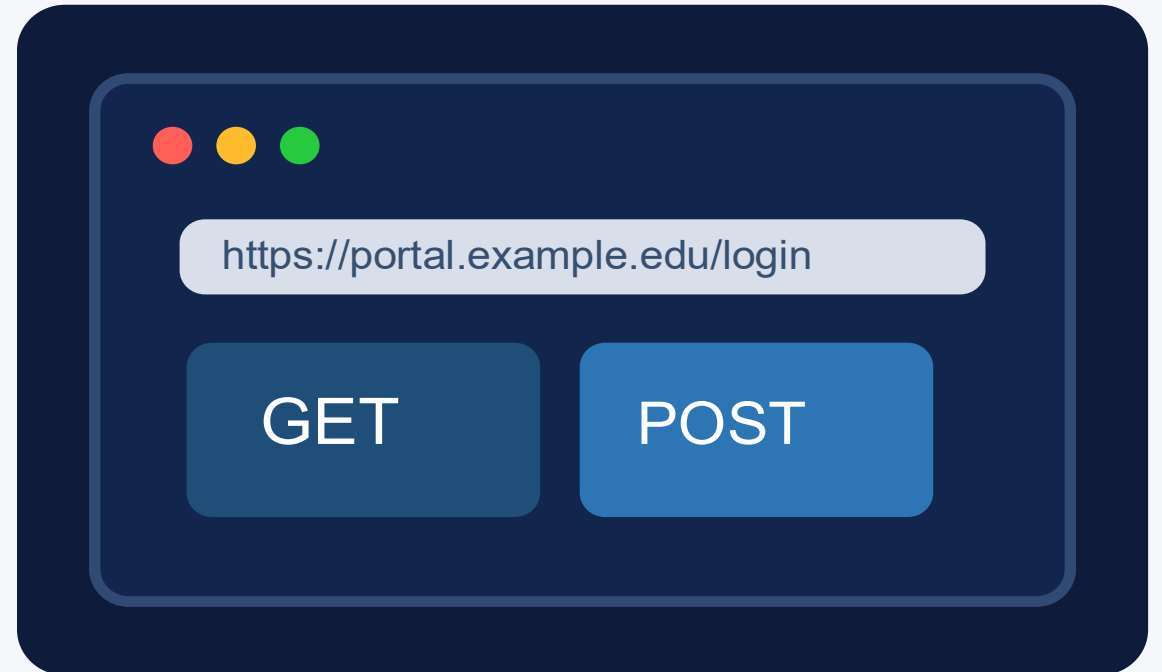
- Define client, server, request, and response.
- Read and analyze a URL into parts.
- Differentiate common HTTP methods such as GET and POST.
- Interpret major HTTP status code groups.
- Relate forms, browsers, and backend validation to real web applications.



What Happens When You Open a Website?

A simple everyday question with a very important backend answer

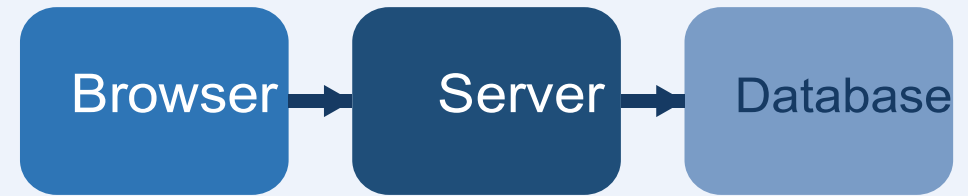
- You type a URL or click a link.
- The browser sends a request over the network.
- A server receives the request and processes it.
- The server returns a response.
- The browser displays the result as a web page or data.



Client and Server

Two sides of every web conversation

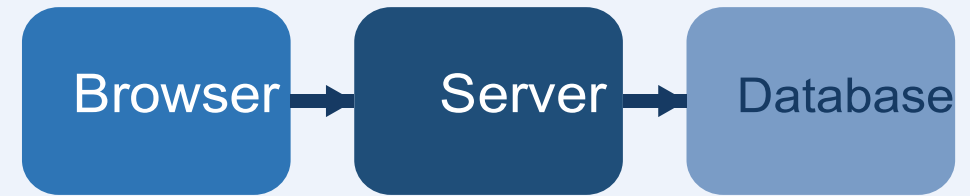
- The client is usually the browser or mobile app.
- The server is the machine or service that listens and responds.
- The client asks; the server answers.
- A web application is built around this repeated conversation.



Frontend, Backend, and Database

The three parts students must keep separate in their minds

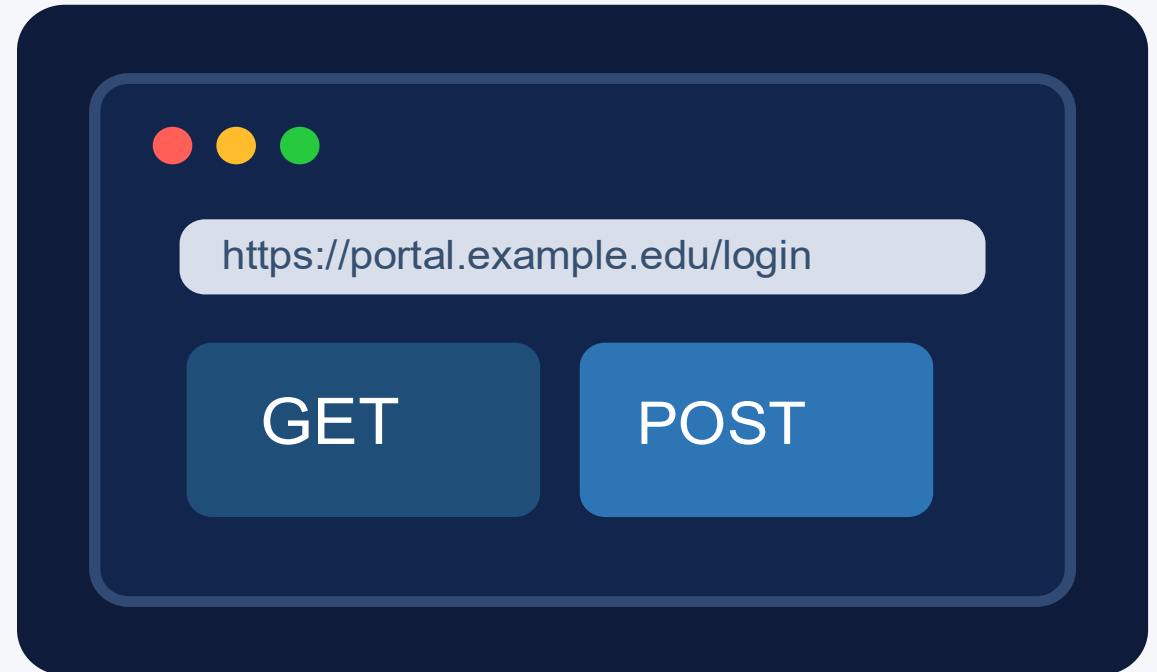
- Frontend: what the user sees and interacts with.
- Backend: logic, validation, permissions, and processing.
- Database: persistent storage for application data.
- These parts cooperate, but each has a different responsibility.



What Does the Browser Do?

Browser responsibilities on the client side

- Shows HTML, CSS, and JavaScript to the user.
- Collects user input from buttons and forms.
- Sends requests to servers.
- Receives responses and renders them.



What Does the Backend Server Do?

The server is where the hidden work happens

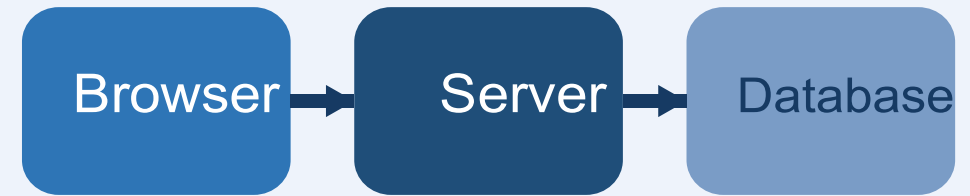
- Receives incoming requests.
- Runs business logic and validation.
- Communicates with the database if needed.
- Returns the correct response: page, JSON, redirect, or error message.



Role of the Database

The memory of the web application

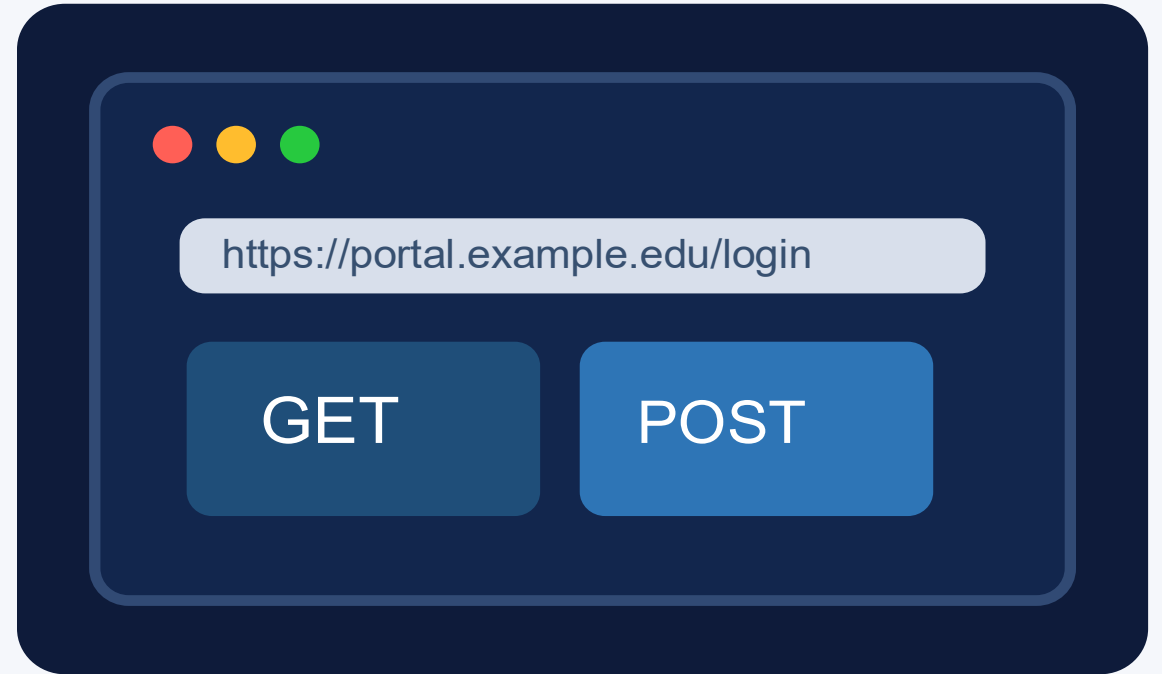
- Stores data permanently.
- Keeps tables such as students, courses, and grades.
- Allows searching, updating, inserting, and deleting records.
- The backend uses it; the browser does not talk to it directly in our basic model.



Static vs Dynamic Websites

Why backend exists in the first place

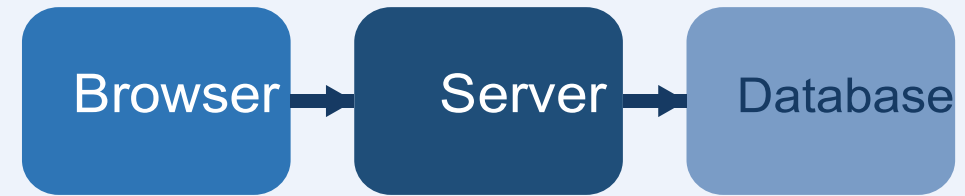
- Static website: same content for every visitor.
- Dynamic website: content changes based on user, data, or action.
- University portals, e-commerce, and dashboards are dynamic.
- Dynamic sites need backend logic and often a database.



The Journey of One Web Request

A request moves through many steps before the user sees a result

- User action: click, submit, or enter URL.
- Browser creates and sends a request.
- Server receives and processes.
- Database may be read or updated.
- Server sends response back to browser.
- Browser renders the result.



What Is a URL?

Uniform Resource Locator - the address of a web resource

- A URL tells the browser where to go.
- It can point to a page, image, API endpoint, or file.
- It includes meaningful parts such as protocol, domain, and path.

`https://example.com:8000/students/list?level`

Protocol

Domain

Port

Path

Query

Parts of a URL

Every section of a URL carries information

- Protocol: http or https
- Domain: example.com
- Port: optional, such as 8000
- Path: /students/list
- Query string: ?level=2&sort=name

https://example.com:8000/students/list?level

Protocol

Domain

Port

Path

Query

Example URL Analysis

Read the URL exactly like a backend developer

- `https://portal.cu.edu.eg:443/students/profile?id=25`
- `https` -> secure protocol
- `portal.cu.edu.eg` -> domain
- `443` -> port
- `/students/profile` -> path
- `?id=25` -> query parameter

`https://example.com:8000/students/list?level`

Protocol

Domain

Port

Path

Query

Path vs Query String

Two very common URL components with different meanings

- The path identifies the general resource or route.
- The query string adds extra details or filters.
- Example path: /courses/list
- Example query: ?department=CS&level=2

https://example.com:8000/students/list?level

Protocol

Domain

Port

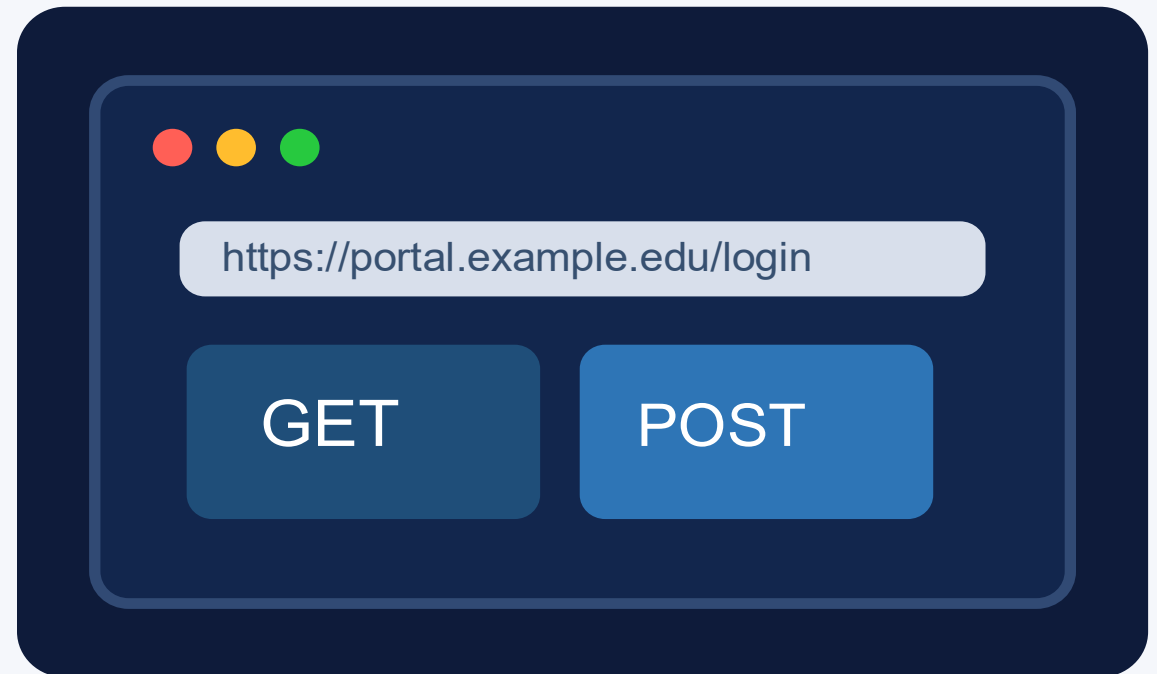
Path

Query

What Is HTTP?

The language used by browsers and web servers

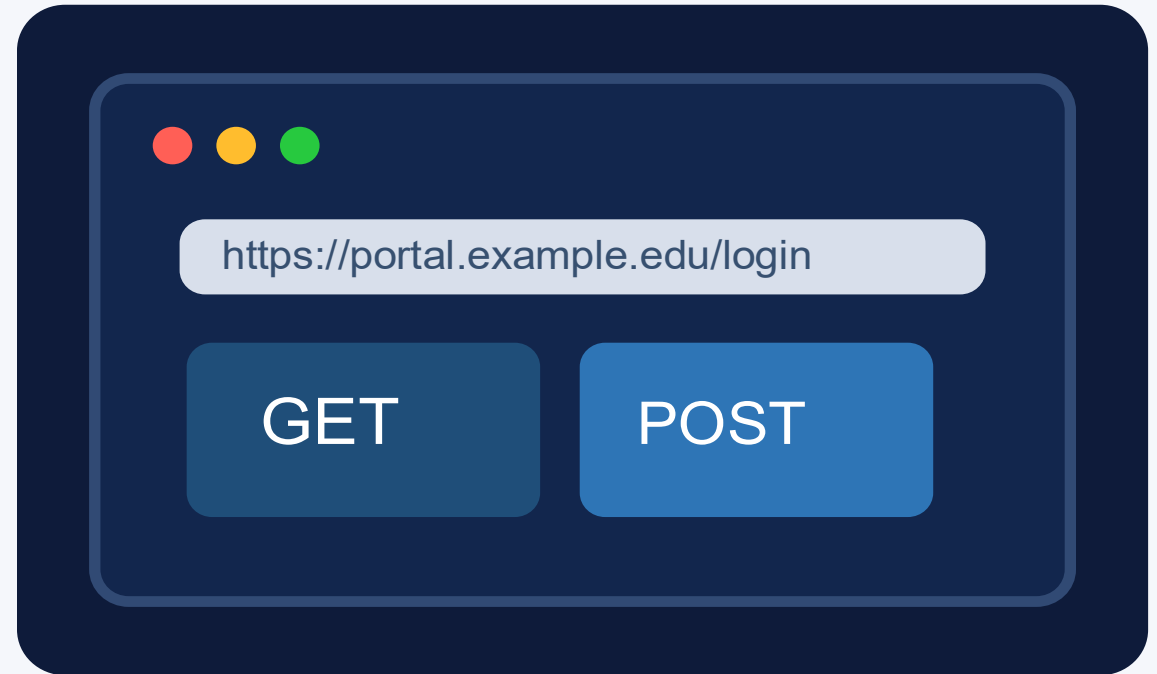
- HTTP stands for HyperText Transfer Protocol.
- It defines how requests and responses are structured.
- It does not decide your business logic; it defines the communication format.



Request: The Message Sent by the Client

A request asks the server to do something or provide something

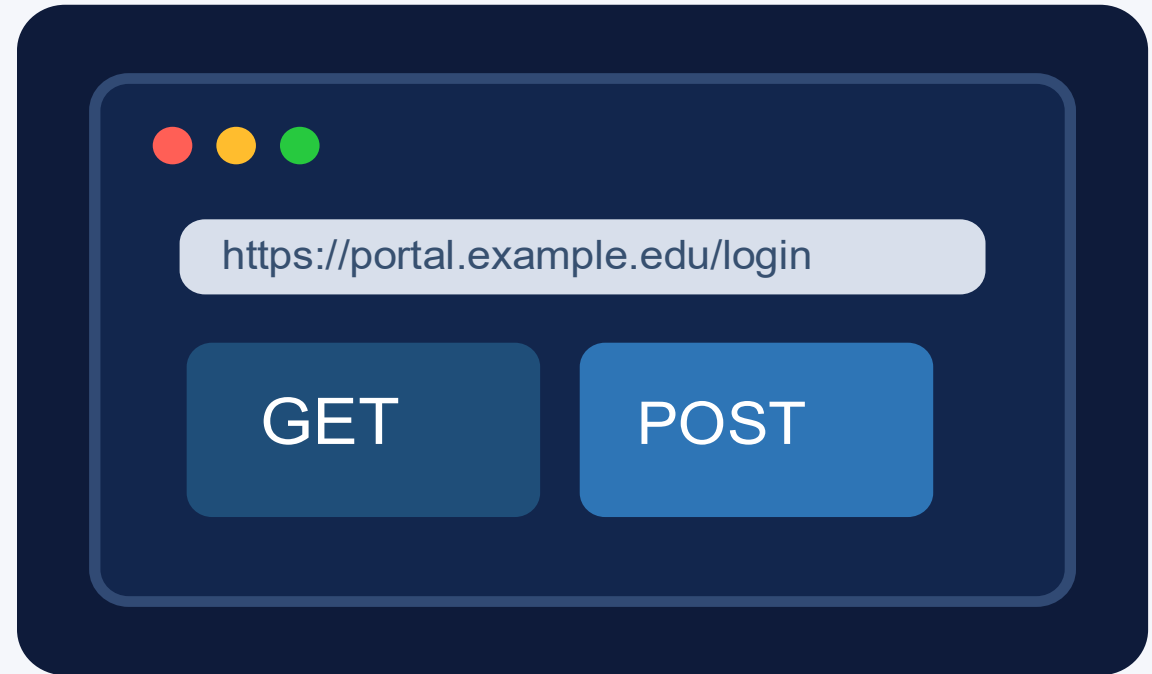
- Contains a method such as GET or POST.
- Contains a target URL or path.
- May contain headers.
- May contain a body with sent data.



Response: The Message Sent by the Server

The response tells the client what happened

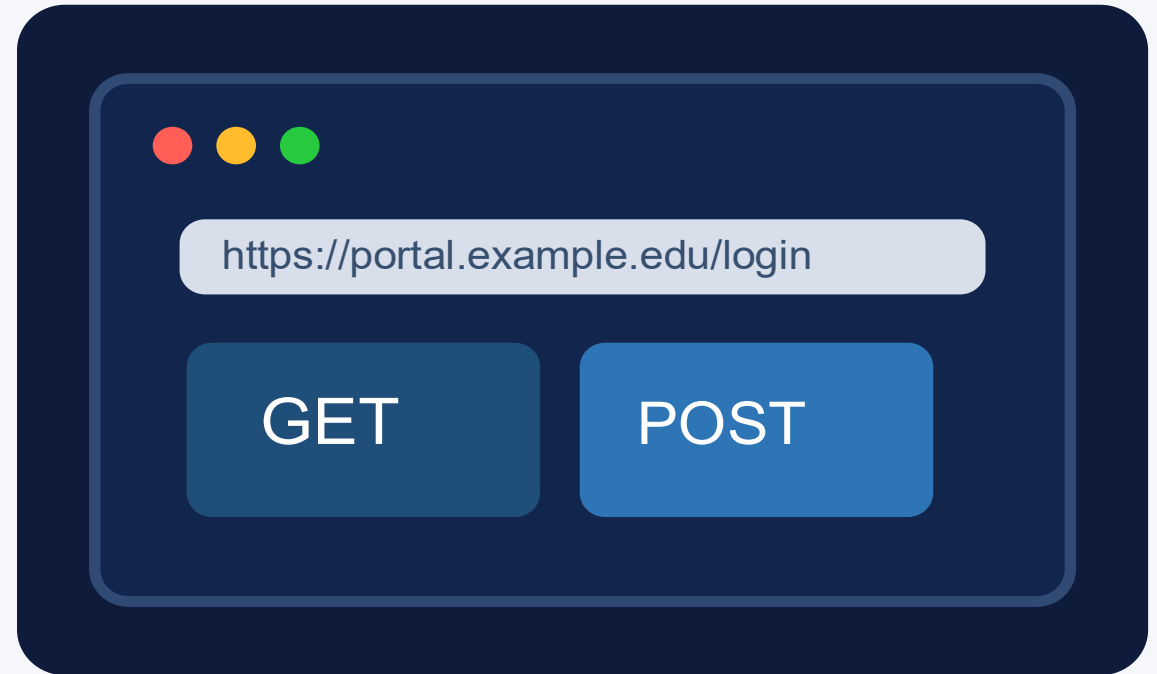
- Contains a status code such as 200 or 404.
- Contains headers that describe the response.
- Contains a body: HTML, JSON, file, or error text.



Headers and Body

Two parts that appear in both requests and responses

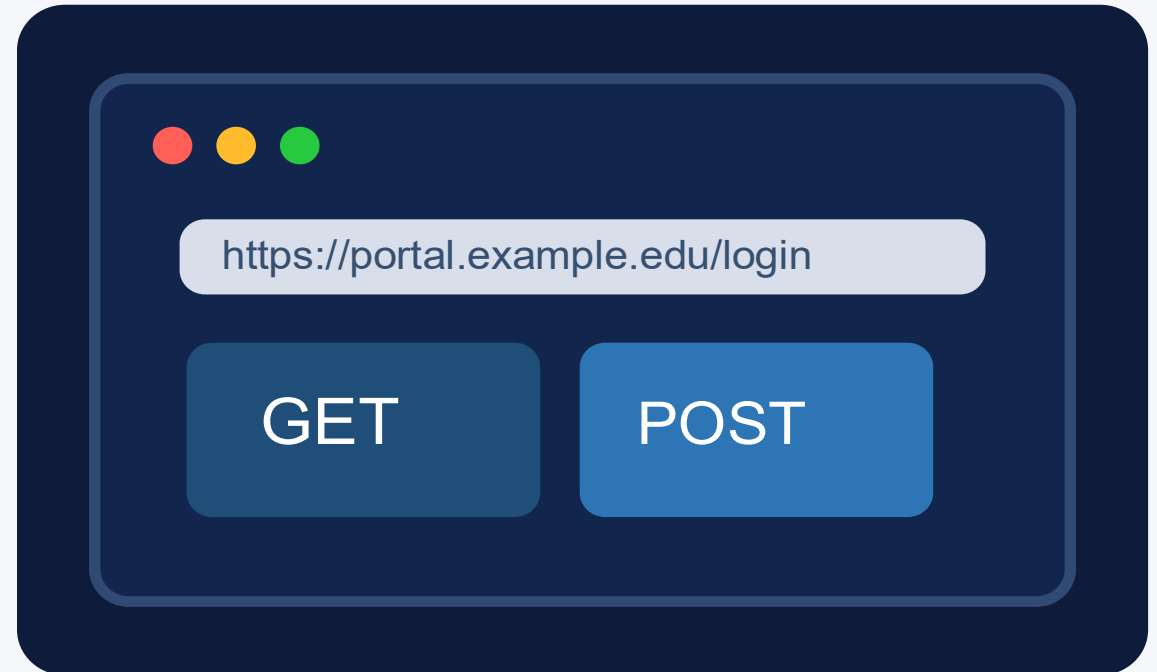
- Headers provide metadata: content type, authorization, length, and more.
- Body carries the actual data when needed.
- Not every request needs a body.
- Many responses do include a body.



HTTP Methods Overview

The method tells the server the intention of the request

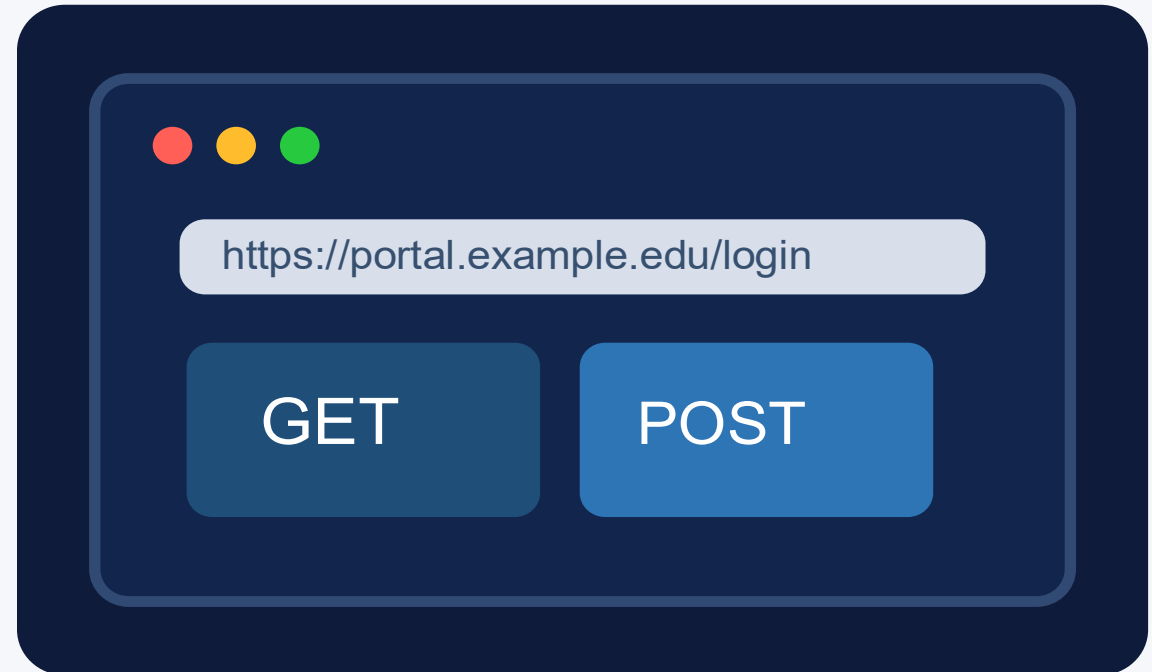
- GET -> read data or page
- POST -> send new data
- PUT/PATCH -> update existing data
- DELETE -> remove data



GET Method

Used to retrieve a page or data without creating a new record

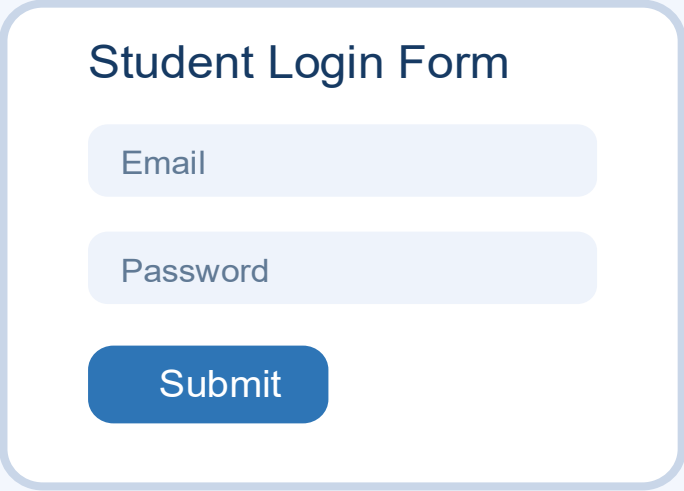
- Common for opening pages and reading lists.
- Usually parameters are visible in the URL.
- Should not be used to create sensitive updates in normal design.



POST Method

Used to send data that the server should process or store

- Common with forms such as login, registration, and create record.
- Data is sent in the request body.
- Used when a new action or new record is required.

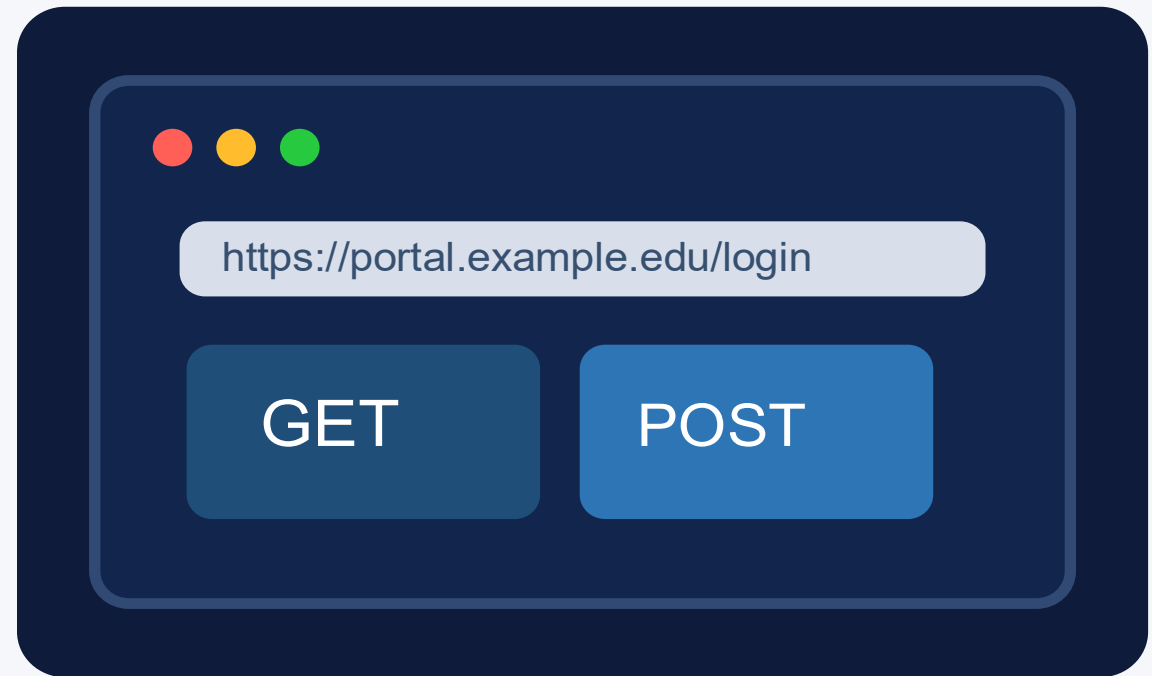


The diagram shows a 'Student Login Form' with a light blue border and rounded corners. Inside the form, there are two light blue input fields: one labeled 'Email' and one labeled 'Password'. Below these fields is a dark blue button with the text 'Submit' in white.

PUT and PATCH

Methods often associated with updating existing data

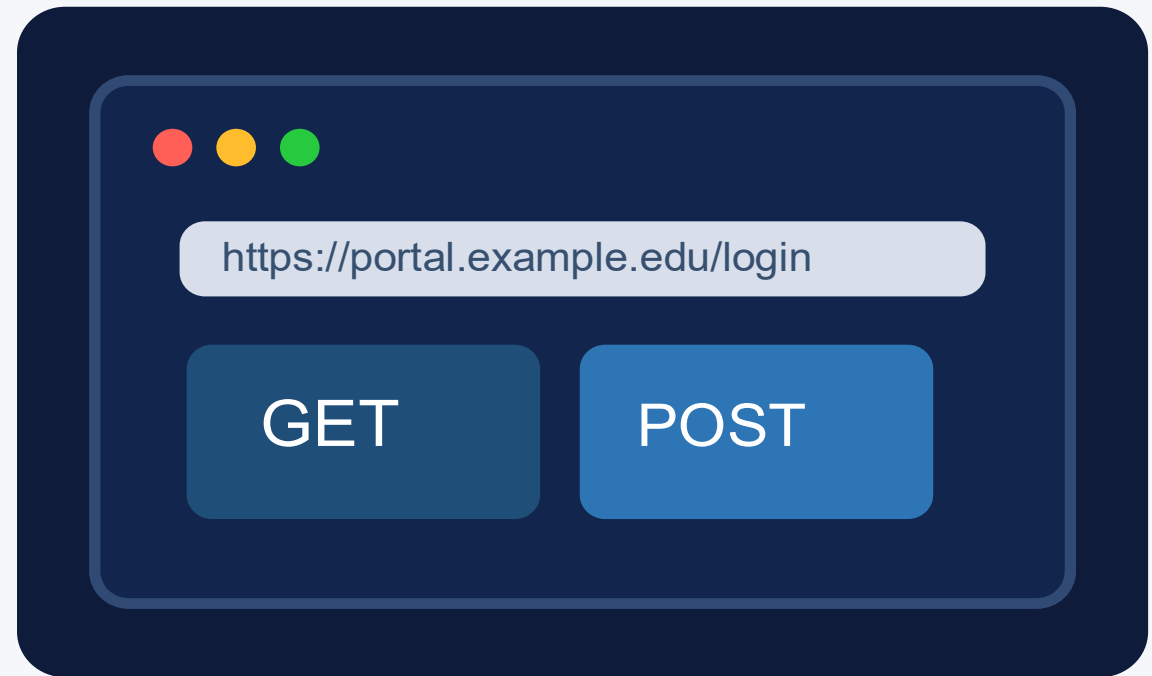
- PUT often means replace the resource fully.
- PATCH often means update only part of it.
- In beginner web apps, the exact usage may be hidden by forms or frameworks, but the idea matters.



DELETE Method

Used when the client asks the server to remove a resource

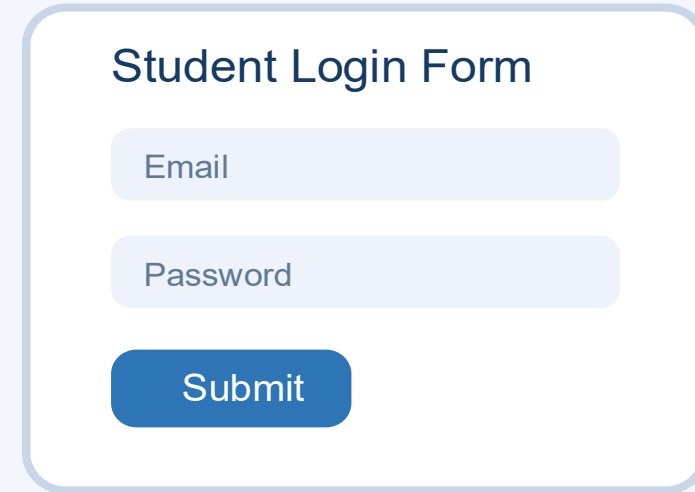
- Represents deleting data on the server side.
- Should be protected by authorization and validation.
- A delete action should not be triggered carelessly by a simple public link.



Safe and Idempotent - Simplified View

Two useful ideas when thinking about HTTP behavior

- GET is usually treated as safe because it should only read.
- Idempotent means repeating the same request should have the same effect.
- DELETE is often idempotent conceptually, even if the first call removes the data.



Student Login Form

Email

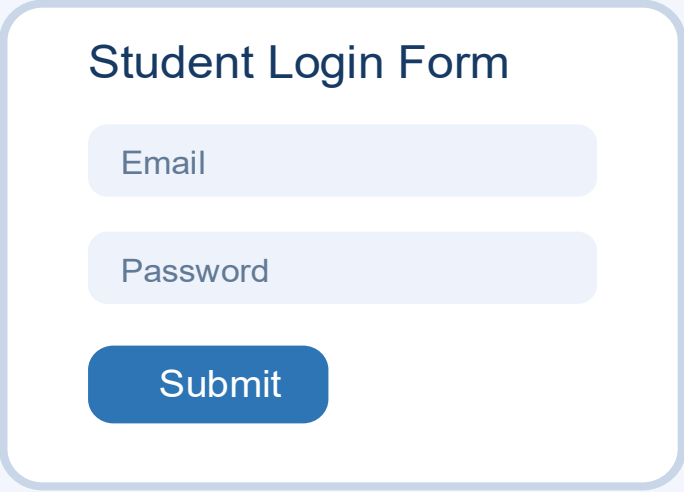
Password

Submit

HTML Forms and Backend Requests

Forms are a common bridge between the user and the server

- A form collects user input.
- The form sends data to a target URL.
- The method is usually GET or POST in basic HTML forms.
- The backend receives the data and acts on it.

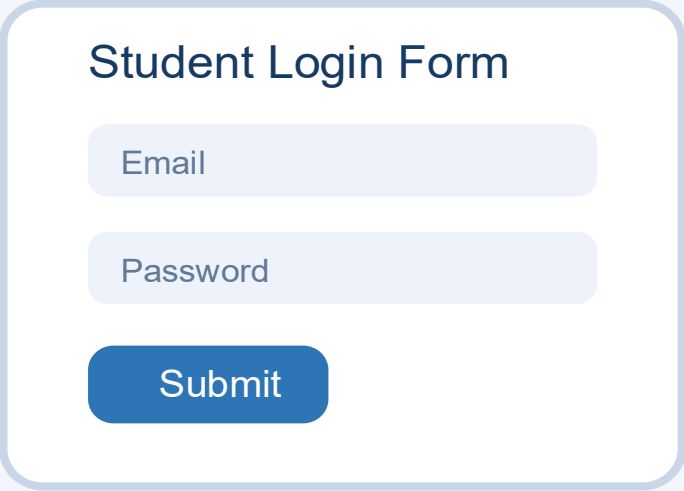


The diagram shows a 'Student Login Form' with a light yellow background and a light blue border. It contains three elements: a text input field labeled 'Email', a text input field labeled 'Password', and a blue button labeled 'Submit'.

Example: Login Form Flow

A practical sequence students already know well

- User enters email and password.
- Browser sends POST request to login endpoint.
- Server validates user credentials.
- Server returns success, failure, or redirect.

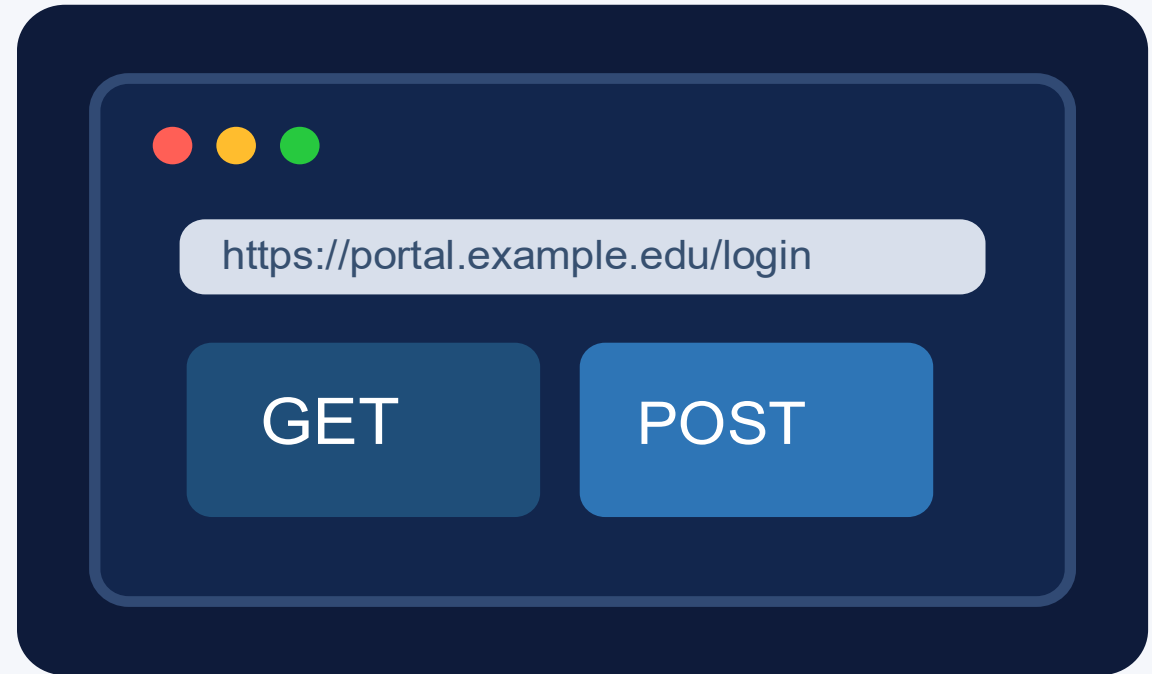


The diagram shows a 'Student Login Form' with a light yellow background and a light blue border. It contains three elements: a text input field labeled 'Email', a text input field labeled 'Password', and a blue 'Submit' button.

Why Backend Validation Is Necessary

Never trust input just because it came from a nice form

- Frontend validation improves user experience.
- Backend validation protects data integrity and security.
- The server must re-check required fields, formats, and permissions.



What Is JSON?

A common format for sending structured data

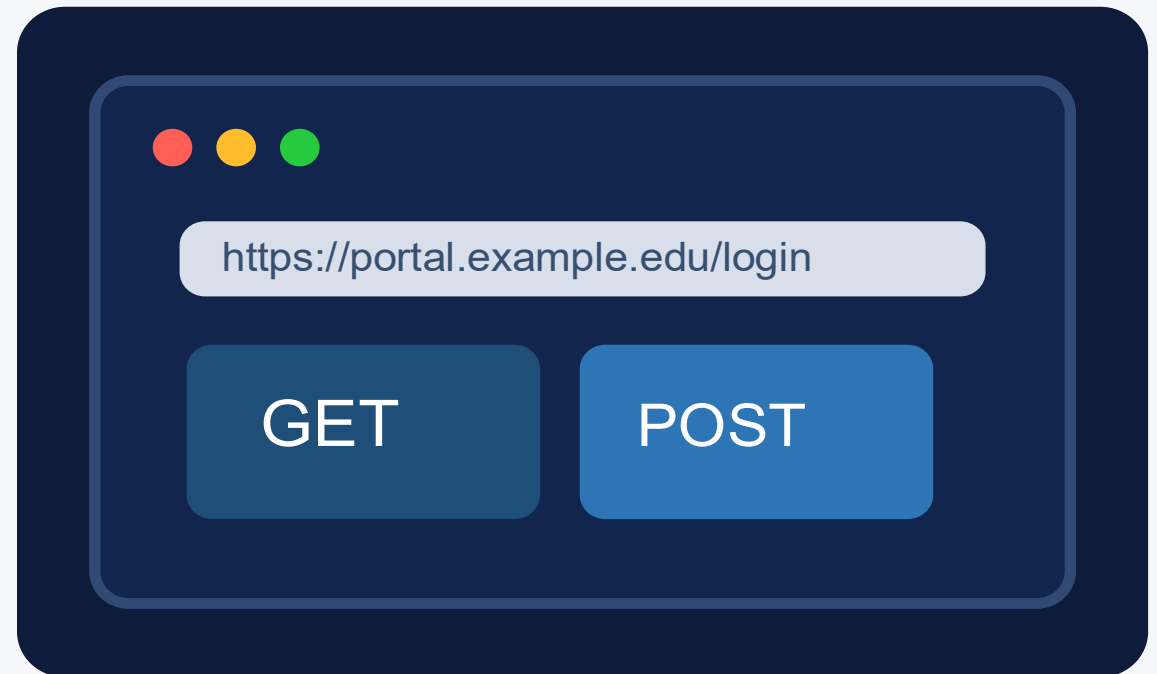
- JSON is text-based structured data.
- Common in APIs and modern web apps.
- Uses key-value pairs and arrays.
- Python can easily read and write JSON.

```
JSON Example
1 {
2   "name": "Ali",
3   "level": 2,
4   "department": "CS"
5 }
```

What Is an API?

A contract for communicating with software through requests and responses

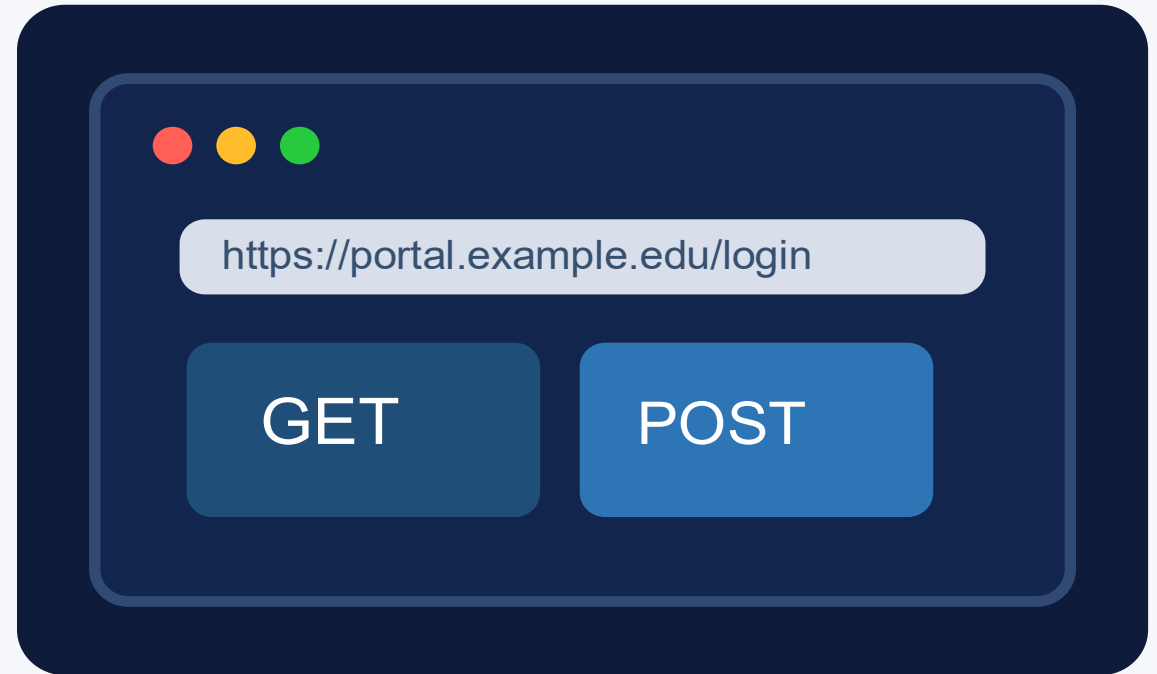
- API stands for Application Programming Interface.
- A web API often returns data instead of full rendered pages.
- The same HTTP principles still apply: URL, method, headers, body, response.



Browser Developer Tools - Network Thinking

A backend developer should learn to observe requests and responses

- Open browser developer tools.
- Use the Network tab to see requests.
- Observe method, status code, and timing.
- This is useful for debugging web applications.



HTTP Status Codes - Big Picture

The server uses status codes to summarize the result of a request

- 1xx -> informational
- 2xx -> success
- 3xx -> redirection
- 4xx -> client-side problem
- 5xx -> server-side problem



1xx Informational

Rare in beginner daily work, but useful to know exists

- This family is for informational responses.
- It is not the main focus in beginner backend courses.
- Know that not every response means final success or final failure immediately.



2xx Success Codes

The server accepted the request successfully

- 200 OK -> successful general response.
- 201 Created -> a new resource was created.
- 204 No Content -> success but no body returned.

1xx

2xx

3xx

4xx

5xx

3xx Redirection Codes

The client is told to go somewhere else

- 301 -> moved permanently.
- 302 -> temporary redirect in many practical contexts.
- Redirects are common after login or after successful form submission.



4xx Client Error Codes

The request has a problem from the client side perspective

- 400 Bad Request -> malformed or invalid request.
- 401 Unauthorized -> authentication required.
- 403 Forbidden -> understood but not allowed.
- 404 Not Found -> target resource not found.



5xx Server Error Codes

The server failed while handling an apparently valid request

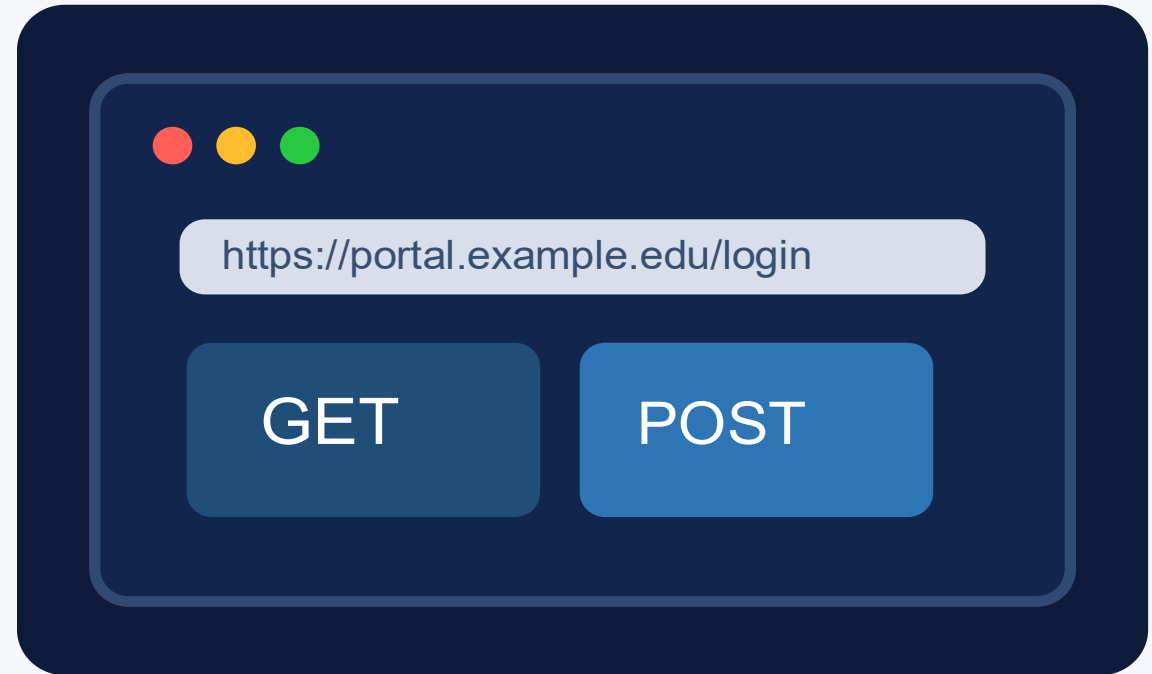
- 500 Internal Server Error -> generic server failure.
- 502 Bad Gateway and 503 Service Unavailable may appear in larger deployments.
- These usually point to backend/server-side issues, not simple wrong URLs.



Real Scenario 1 - Opening a Valid Page

A normal successful browsing case

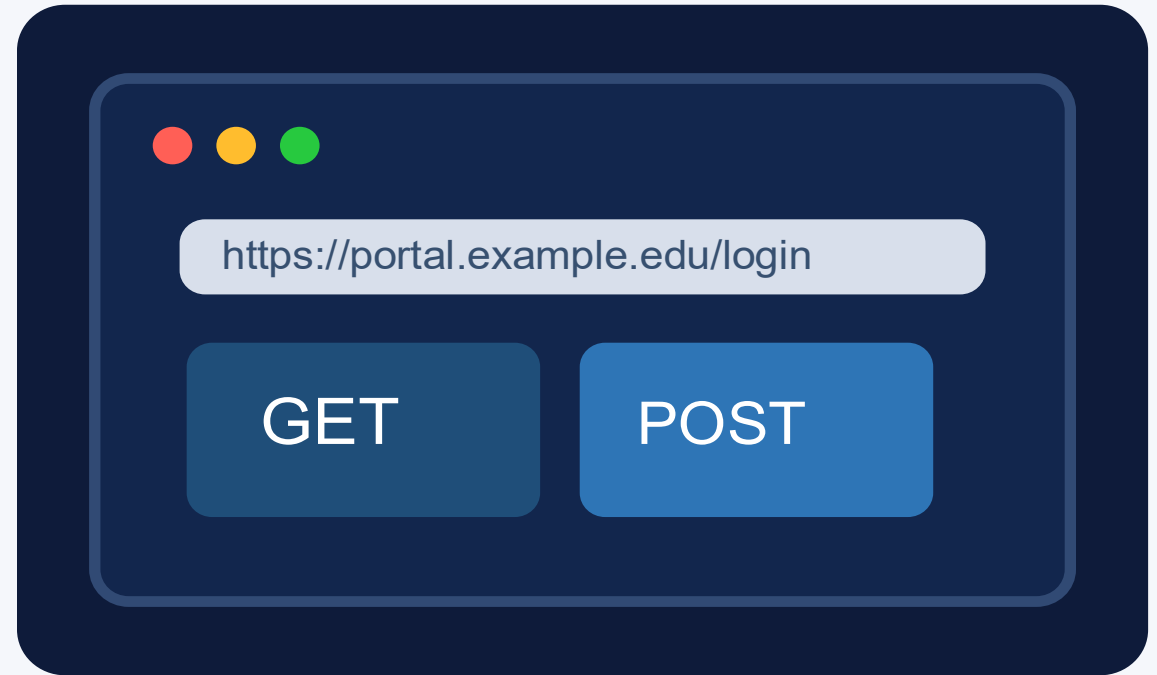
- User opens /courses/list
- Server finds the route and prepares data
- Response usually returns 200 OK
- Browser renders the page



Real Scenario 2 - Page Not Found

A classic 404 example

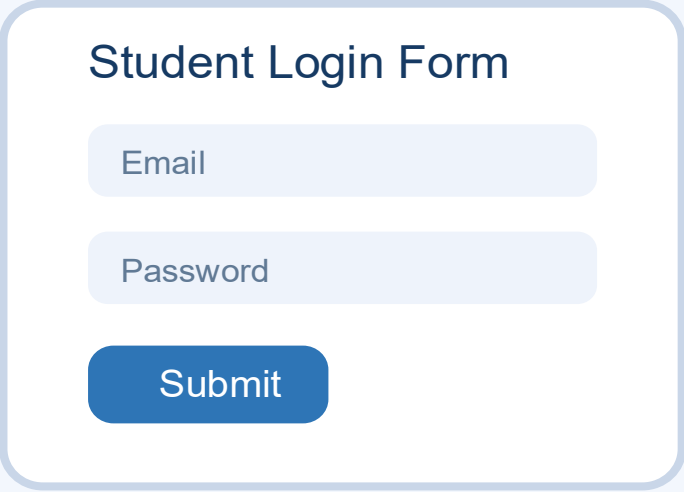
- User opens a wrong or missing path.
- Server does not find matching route or resource.
- Response returns 404 Not Found.



Real Scenario 3 - Forbidden Action

A 403 example based on permissions

- A normal student tries to open an admin-only page.
- Server understands the request.
- Server refuses because permissions are missing.
- Response can be 403 Forbidden.



The diagram shows a 'Student Login Form' with a light blue border and rounded corners. Inside the form, there are two light blue input fields: one labeled 'Email' and one labeled 'Password'. Below these fields is a dark blue button with the text 'Submit' in white.

Complete Example - Login Request and Response

Connect methods, status codes, validation, and redirect

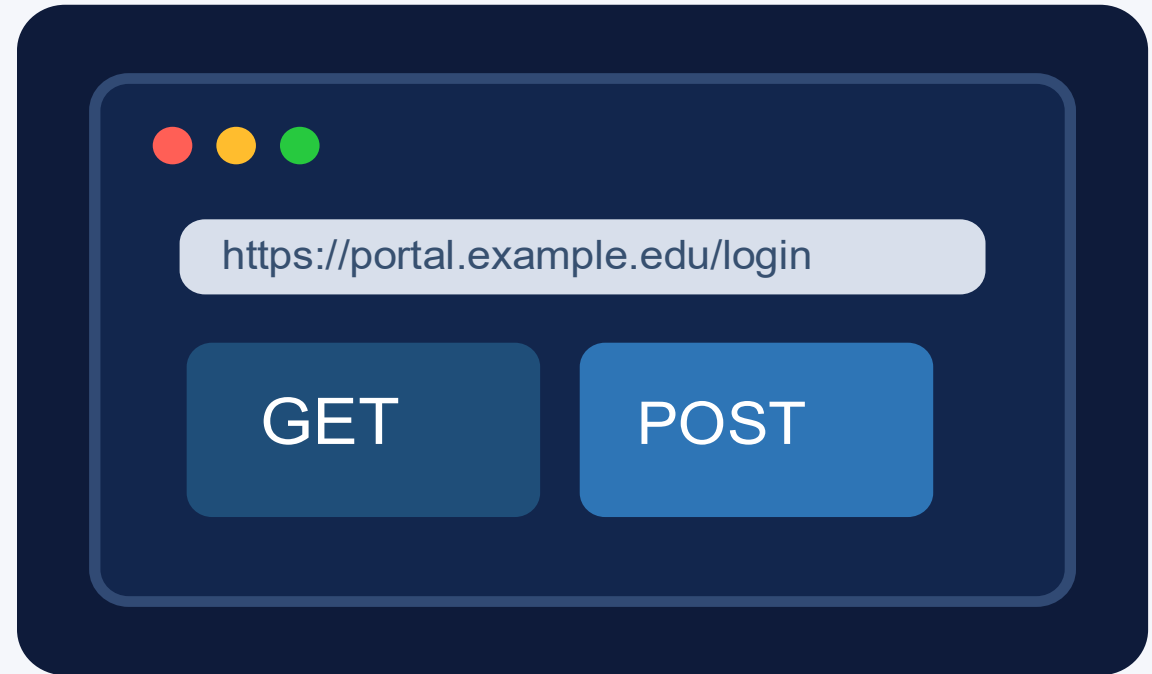
- User submits login form with POST.
- Server validates email and password.
- If successful -> create session and redirect.
- If failed -> return form again with error message.

The diagram shows a rounded rectangular box representing a login form. Inside the box, at the top, is the title 'Student Login Form'. Below the title are two light blue input fields. The first field is labeled 'Email' and the second is labeled 'Password'. Below these fields is a dark blue button with the text 'Submit' in white.

Cookies and Sessions - Intro Only

How the server remembers that a user is logged in

- A cookie is small data stored in the browser.
- A session represents server-side remembered user state.
- After login, the server often needs a way to recognize the same user later.



Example - Course Registration Flow

A familiar university case

- Student opens registration page with GET.
- Student chooses course and submits form with POST.
- Server checks rules and prerequisites.
- Database stores registration if valid.
- Server returns success or error message.

Student Login Form

URL Routing - Preview for Django

The backend maps paths to logic

- A route connects a URL path to backend code.
- Different paths lead to different views or handlers.
- Later in Django, urls.py will manage this mapping.

https://example.com:8000/students/list?level

Protocol

Domain

Port

Path

Query

HTML Response vs JSON Response

Two common response styles from the backend

- HTML response is meant to be rendered as a page.
- JSON response is meant to be consumed as data.
- Traditional Django pages often return HTML.
- APIs often return JSON.

Response Example

```
1 HTTP/1.1 200 OK
2 Content-Type: application/json
3
4 {"status": "success"}
```

Local Development Address

Why we often use localhost:8000 in class

- localhost means this same machine.
- 8000 is a port number commonly used in development.
- When Django runs locally, the browser talks to your own computer.

`https://example.com:8000/students/list?level`

Protocol

Domain

Port

Path

Query

Terminal Review for Web Work

Simple commands help us verify tools and move around projects

- `python --version`
- `pip --version`
- `cd project_folder`
- `dir` or `ls` to list files

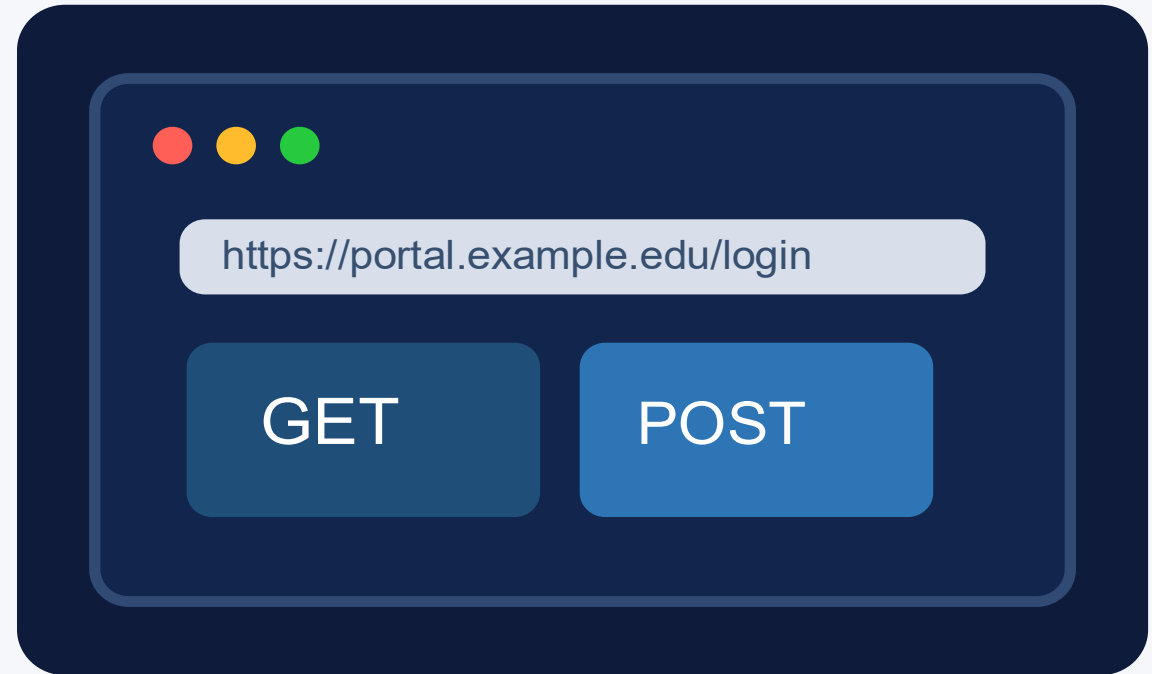
A dark-themed terminal window with a title bar containing three colored circles (red, yellow, green) and the text "Terminal Commands". The terminal displays four numbered lines of code:

```
1 python --version
2 pip --version
3 cd backend_project
4 dir
```

Quick Check: Is Python Ready?

Environment checks before running web code

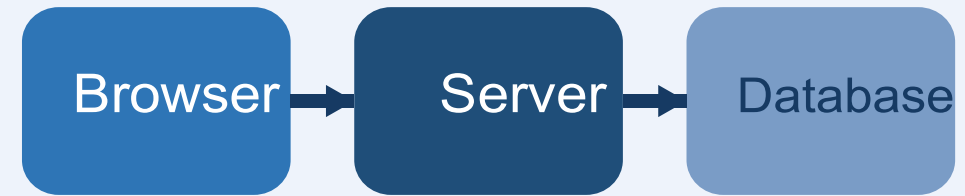
- Install Python first.
- Make sure the python command works.
- Make sure pip works.
- Use VS Code terminal or system terminal to test.



A Very Simple Local Server Idea

Even before Django, browsers can talk to a local server conceptually

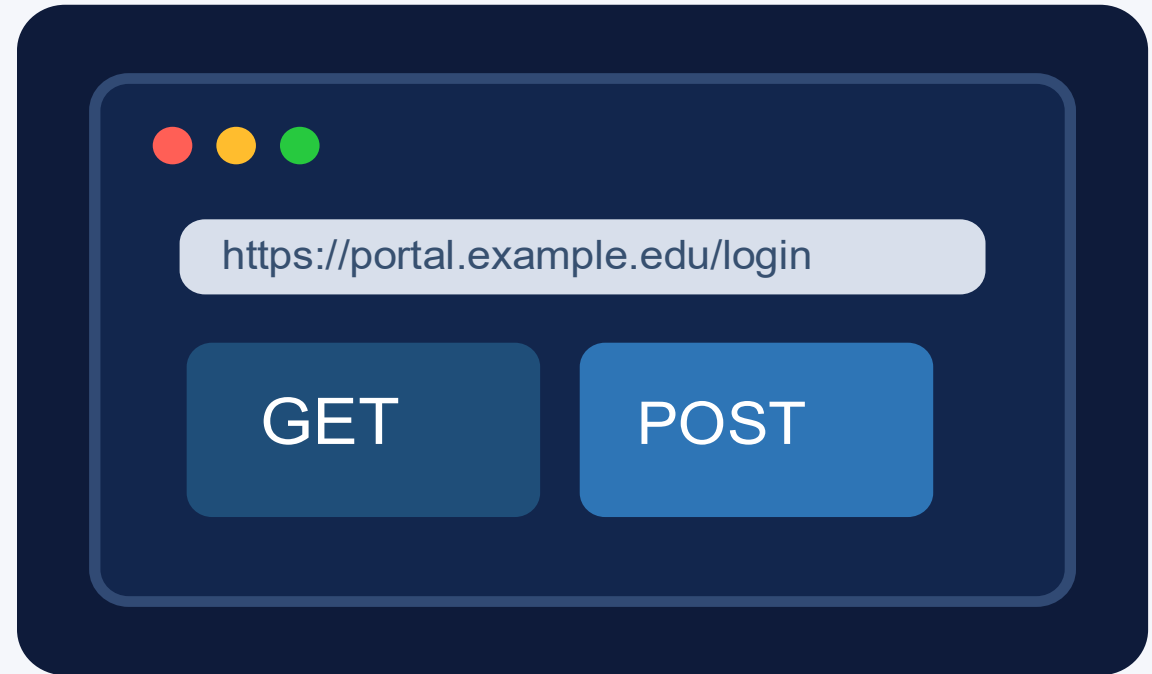
- A browser needs an address and a listening service.
- Frameworks like Django make this organized and powerful.
- The principle remains: request goes in, response comes out.



Common Beginner Mistakes in Web Basics

Confusions that must be fixed early

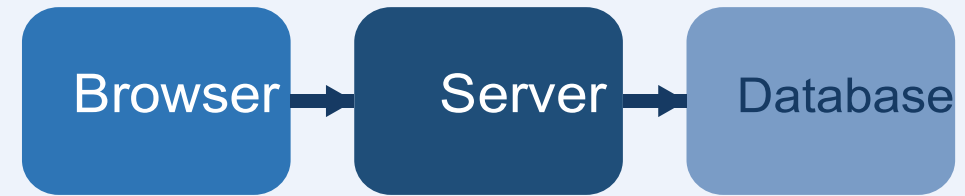
- Thinking frontend and backend are the same.
- Using GET when POST is more suitable.
- Not understanding why 404 and 500 are different.
- Believing frontend validation alone is enough.
- Confusing path and query string.



Practice 1 - Classify the Component

Is it frontend, backend, or database?

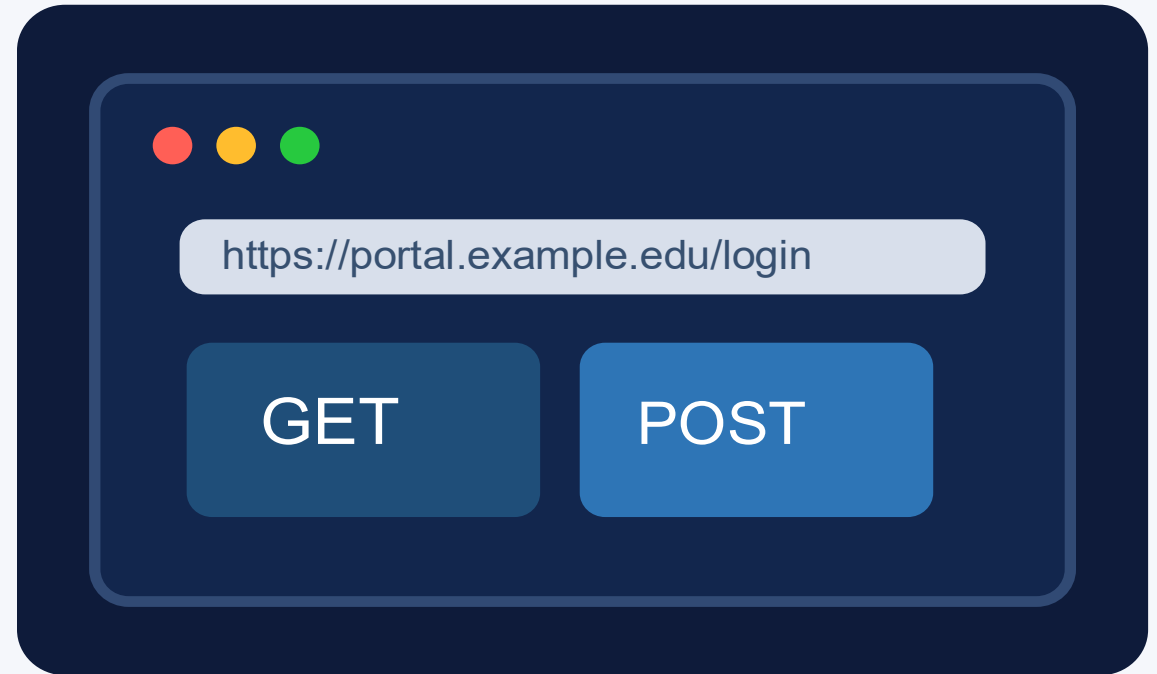
- Displaying a button -> ?
- Checking password correctness -> ?
- Saving a grade record -> ?
- Showing a success message -> ?



Practice 2 - Choose the HTTP Method

Select the most suitable method for each action

- Open the students list.
- Submit login credentials.
- Update student phone number.
- Delete a course registration.



Practice 3 - Analyze This URL

Read the address like a professional

- `http://localhost:8000/courses/list?level=2&semester=1`
- Find the protocol.
- Find the host.
- Find the port.
- Find the path.
- Find the query string.

`https://example.com:8000/students/list?level`

Protocol

Domain

Port

Path

Query

Practice 4 - Which Status Code Fits?

Match the event with the likely code

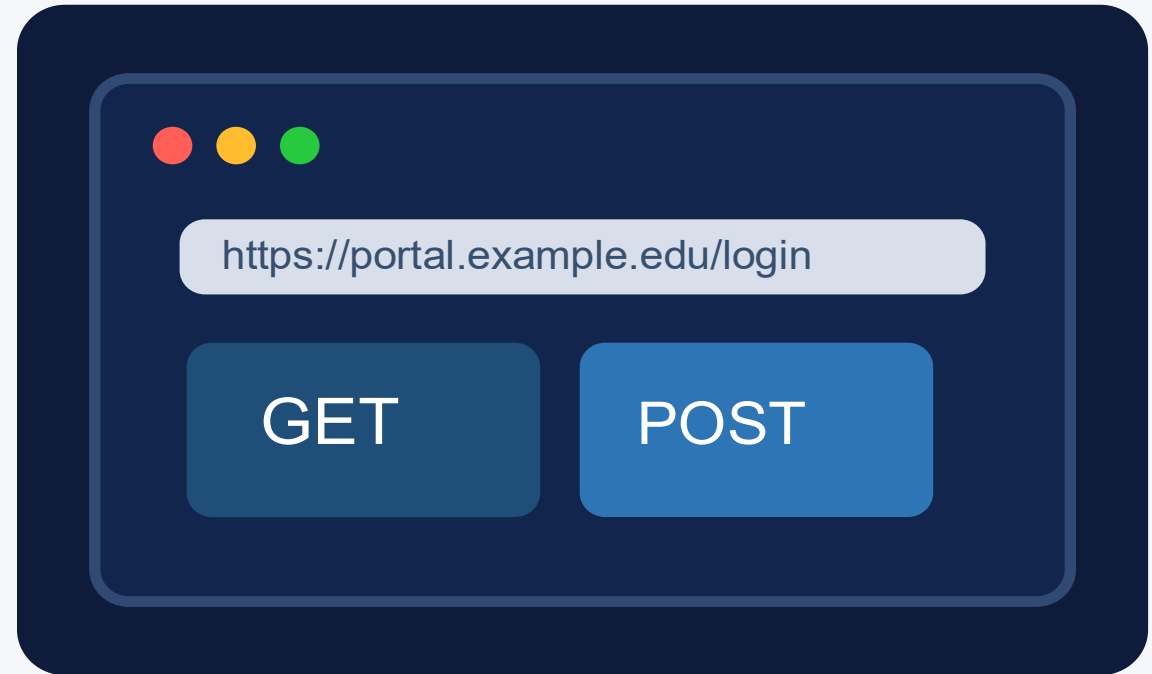
- Valid page opened successfully.
- Requested route does not exist.
- User is not allowed to access admin page.
- Unexpected server bug happened.



Mini Quiz 1

Quick oral questions

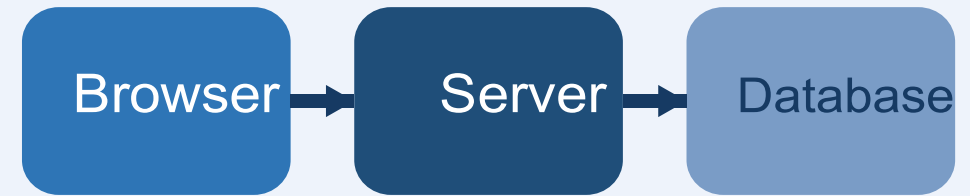
- What is the difference between a request and a response?
- What is the difference between path and query string?
- Why is backend validation necessary even when frontend validation exists?



Mini Quiz 2

Continue reinforcing the key ideas

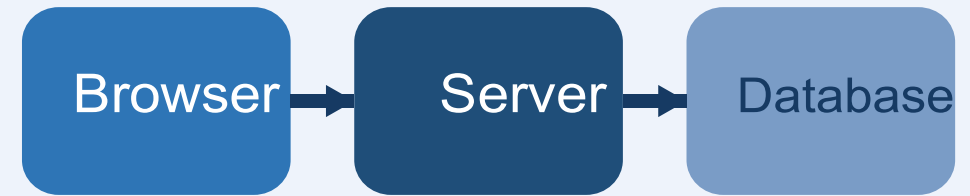
- Which method is usually used to open a page?
- Which code often means page not found?
- Which component stores data permanently?
- What does localhost mean?



Lecture Map - The Whole Story in One View

A mental summary before closing

- URL gives the address.
- HTTP organizes the conversation.
- Method gives the intention.
- Server processes and validates.
- Status code summarizes the result.
- Browser shows the final output.



Most Important Takeaways

The ideas students must remember after class

- A web app is a conversation between client and server.
- Backend is responsible for logic, validation, and data access.
- GET and POST are not interchangeable.
- 404 and 500 do not mean the same thing.
- URL structure matters.



How This Connects to Django

Why today's lecture is essential before framework work

- Django uses routes for URLs.
- Django views handle requests and return responses.
- Django forms send GET/POST data.
- Django apps often return 200, 302, 404, and other status codes.

`https://example.com:8000/students/list?level`

Protocol

Domain

Port

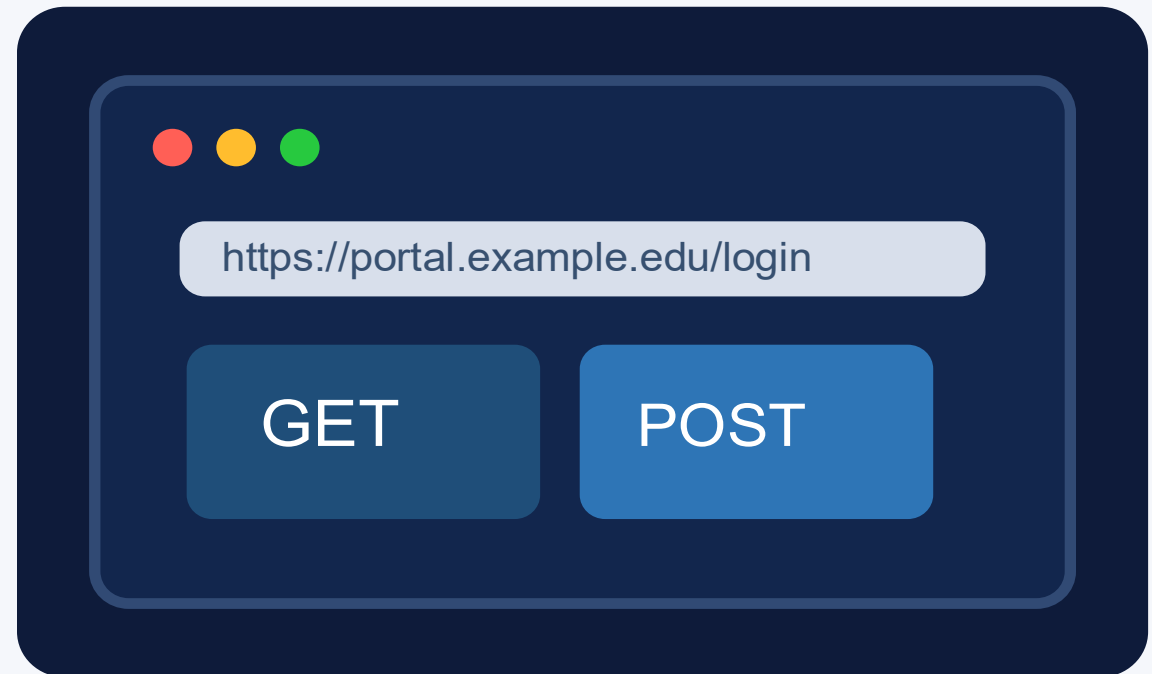
Path

Query

Homework

Short tasks to strengthen understanding before the next lecture

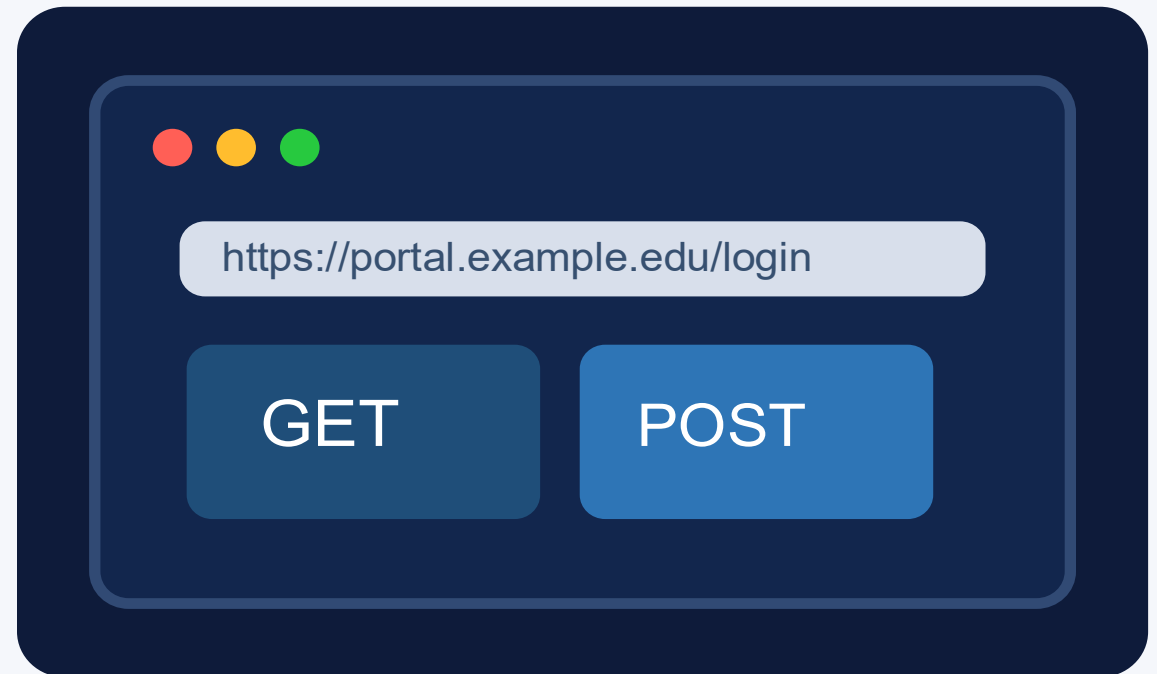
- Write 5 examples of URLs and label their parts.
- Give 3 examples where GET is suitable and 3 where POST is suitable.
- Write the meaning of 200, 403, 404, and 500 in your own words.
- Observe one website using browser developer tools if possible.



Preview of Lecture 5

Next step: starting Django project setup

- What is Django in practice?
- Installing Django with pip.
- Creating a Django project.
- Running the development server.
- Understanding the project structure.



Comparing GET and POST Side by Side

Same endpoint idea, different intent

- GET is mainly for reading.
- POST is mainly for sending data to be processed.
- GET parameters often appear in the URL.
- POST data is usually carried in the body.

Teaching Example

```
1 # Example idea
2 GET  /students/list?page=2
3 POST /students/create
4
5 # One reads, one creates
```

401 vs 403 - A Common Source of Confusion

Authentication issue or permission issue?

- 401 often means the system expects authentication.
- 403 often means identity may be known, but access is denied.
- The exact implementation may differ, but the distinction is useful.

Teaching Example

```
1 401 -> Who are you?  
2 403 -> I know you, but you are not allowed  
here.
```

API Example - JSON Instead of a Full Page

Backend can return data directly

- Request: GET /api/students/1
- Response body may be JSON.
- Useful for mobile apps and JavaScript frontends.
- Same HTTP concepts still apply.

Teaching Example

```
1 {  
2   "id": 1,  
3   "name": "Mona",  
4   "level": 2  
5 }
```

Why Redirect Happens After Forms

A clean browser experience after submission

- Redirect can move the user to a success page.
- Redirect can avoid accidental resubmission on refresh.
- Very common after login or successful create action.

Teaching Example

```
1 POST /login -> success  
2 302 Redirect -> /dashboard/
```


Debugging Question - 404 or 500?

Train yourself to ask the right first question

- If the path is wrong or route missing -> think 404.
- If the code crashed while processing -> think 500.
- Different errors suggest different debugging directions.

Teaching Example

```
1 Wrong URL?      -> 404  
2 Code exception? -> 500
```